

Unified-Core Architecture

Hardware-Sharing Optimization Strategies in a Multi-Format Matrix Engine



Department name: Wireless Lab

Authors' name: Simone Machetti, David Guevorkian

Date: August 3, 2026



Matrix multiplication – one kernel, two industries

$$C_{m,n} = \sum_k A_{m,k} \cdot B_{k,n}$$

- A grid of independent **multiply-accumulate** chains — 2·M·K·N operations, fully parallel, fully regular
- **AI** — it is the algorithm: linear projections, feed-forward layers, attention, convolution lowered to **GEMM**. Operands are aggressively quantized: **4-, 8-, 16-bit integers**, and **block floating point** where dynamic range must be preserved
- **Wireless** — MIMO detection, beamforming and precoding, channel estimation. The same product, but **complex-valued**, and less tolerant of quantization error
- **One engine for both** → integer and complex arithmetic, across several precisions

Optimizing the kernel – can we share hardware?

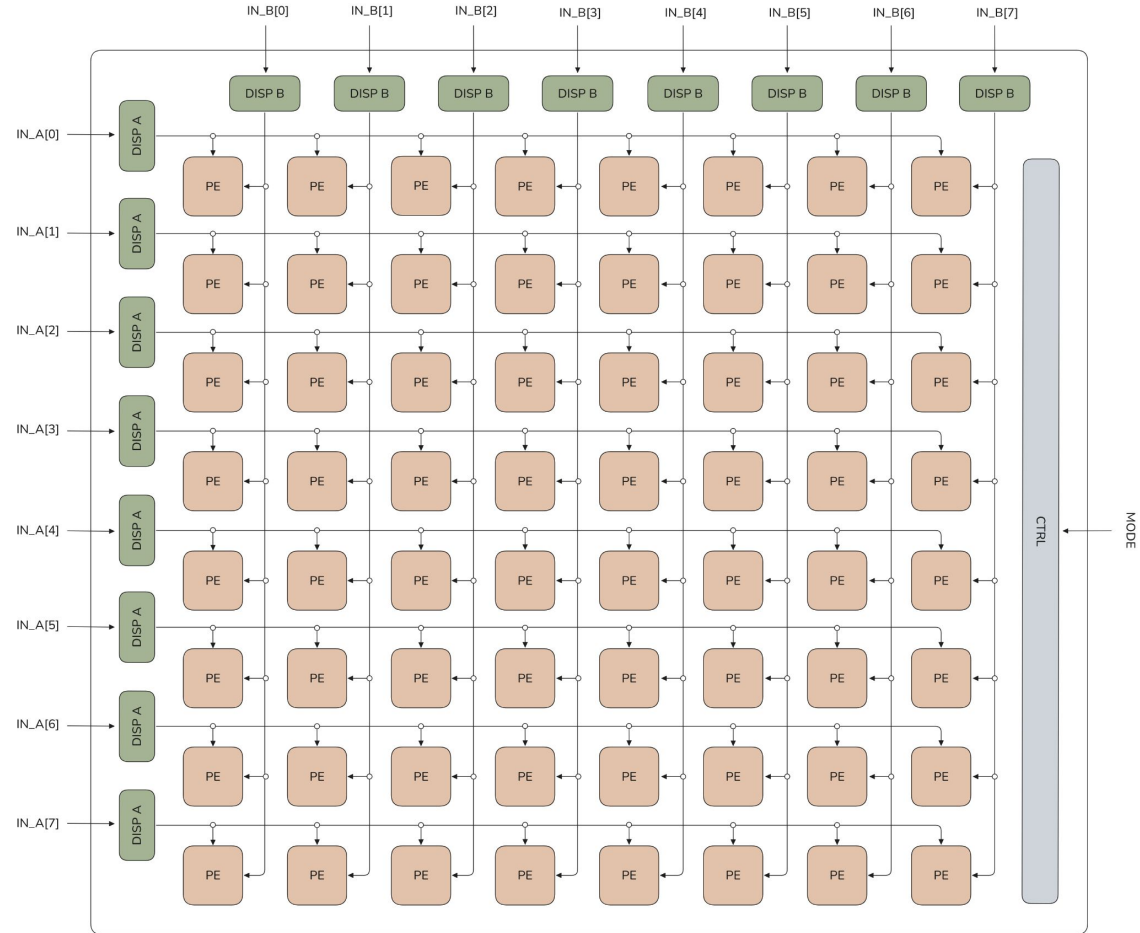
- GEMM dominates these workloads, so any per-operation saving multiplies across the whole system
- In our reference grid, the PEs are ~96 % of both area and power — everything else is noise.
- A spatial array replicates PE units N^2 times, so:

Hardware moved out of the PE and shared is paid $O(N)$ times instead of $O(N^2)$!

- If part of the computation depends on one operand only, it can be moved out of the PE and amortized across rows or columns
- **The question:** can GEMM be re-engineered this way — and does the shared hardware cost less than what it removes?

Reference design – an NxN grid of processing elements

- N^2 PEs, one tile product per PE
- Operand A[r] is broadcast to each row
- Operand B[c] is broadcast to each column
- $PE[r][c]$ computes $A[r] \cdot B[c]$
- Each $PE[r][c]$ holds its own accumulator
- 3 pipeline stages
 - Dispatch & Compute
 - Compress
 - Accumulate



From a large **matrix** product to the **grid**

Steps

1. **Tile** — A large $M \times K \times N$ product is split into tiles sized to what one PE evaluates in one cycle
2. **Pack** — Tile's elements are placed into two fixed-width operand words: **256-bit A** and **256-bit B**
3. **Feed** — **A operands** are shared **per rows** and **B operands** are shared **per columns**
4. **Accumulate** — A large K dimension is streamed as successive tiles into each PE's accumulator

Supported formats – the constraint on any optimization

Family	Used by	Note
Integer	AI	Mixed precision for A and B
Complex integer	Wireless	Four real products each complex one
Block floating point (BFP)	AI	One exponent shared per block
Complex BFP	Both	The two combined

- These expand into **11 operating modes**, all reduced to a single narrow dot-product primitive
- Every optimization **must support all of them** — this is the bar the proposals are measured against

Opt 1 – Replacing the multiplier with a squarer

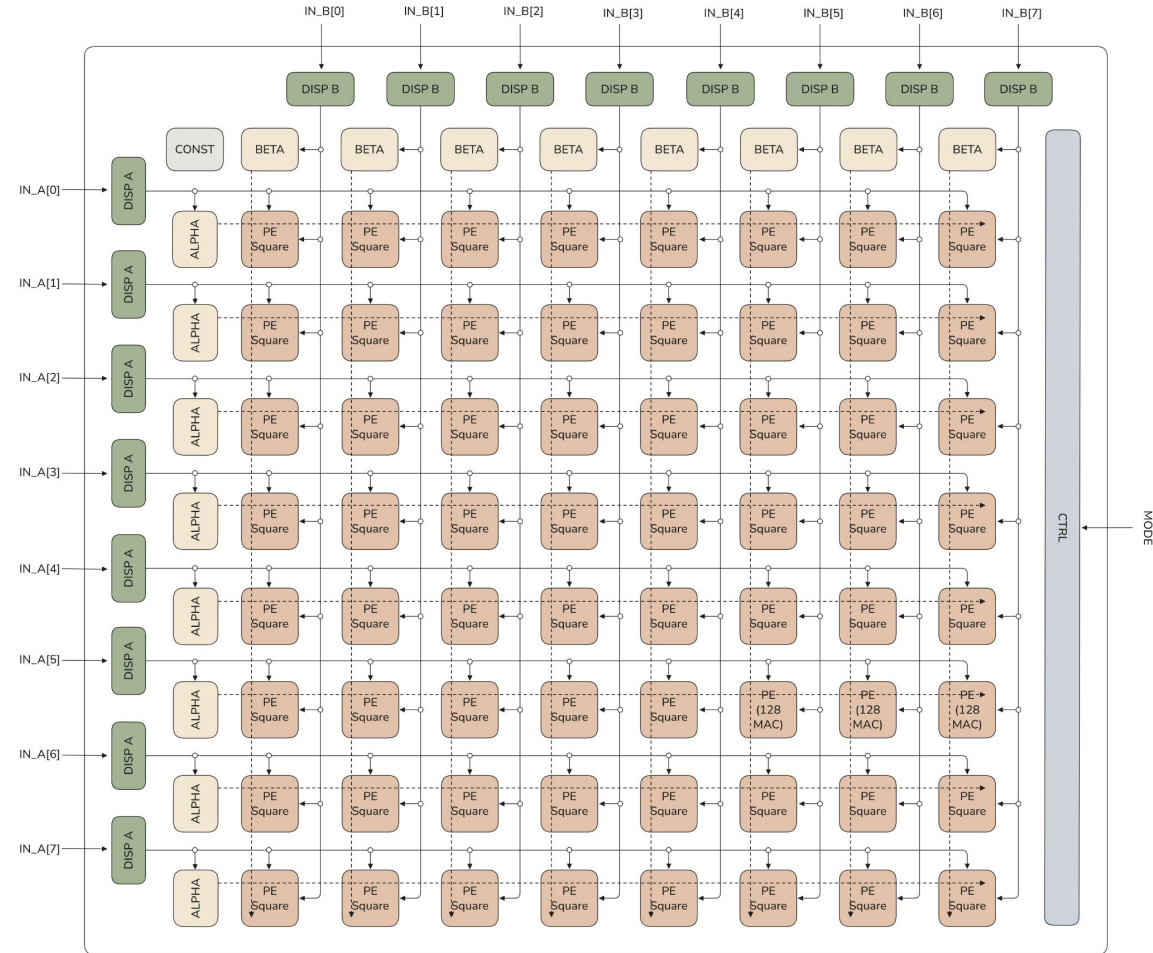
$$a \cdot b = \frac{1}{2} [(a + b)^2 - a^2 - b^2]$$

Term	Depends on	Where it lives	Instances
$(a+b)^2$	A and B	Inside the PE – a squarer replaces the Booth	N^2
a^2	A only	Moved out – shared by each row	N
b^2	B only	Moved out – shared by each column	N

- An $N \times N$ grid needs N^2 squarers but only $N \alpha + N \beta$ generators
- The **saving** scales as N^2 — the added **cost** scales as N

The **square** grid of processing elements

- PE[r][c] computes $(A[r] + B[c])^2$
- Alpha $\alpha[r]$ is broadcast to each row
- Beta $\beta[c]$ is broadcast to each column
- Each PE[r][c] holds its own accumulator
- Constants are added as correction terms
- Still 3 pipeline stages
 - Dispatch & Compute
 - Compress
 - Accumulate



Experimental setup

Stimulus

- Uniformly distributed random numbers
- Full pipeline throughput — fresh A and B every clock cycle
- All rows and columns active
- All 11 modes exercised
- Dynamic power estimated with switching activity annotation onto the netlist

Technology

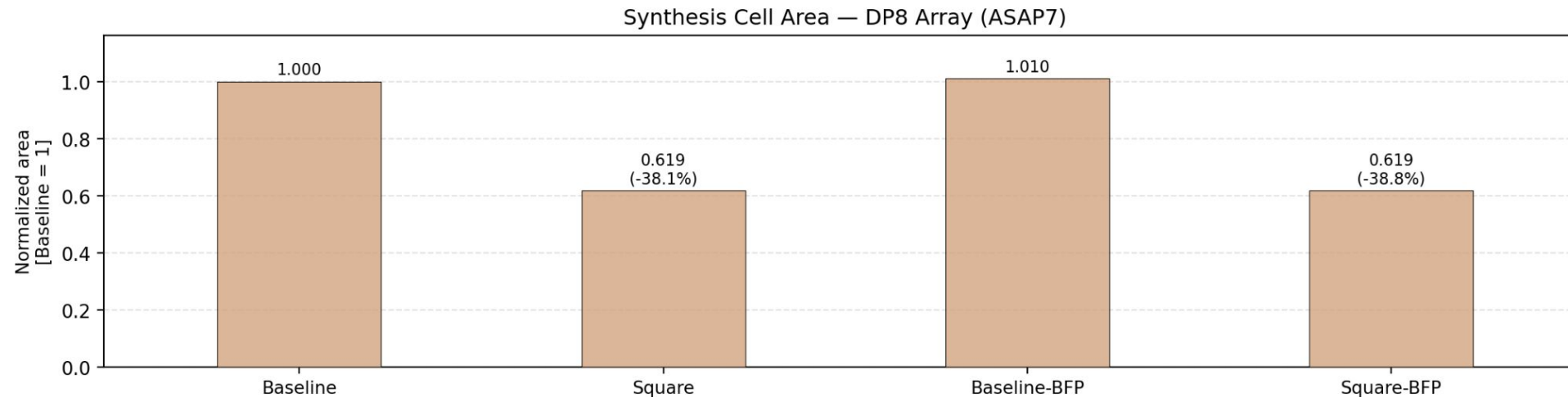
- ASAP7, open-source 7 nm predictive PDK

Tools — fully open-source

Step	Tool
Simulation	Verilator
Synthesis	Yosys
Timing	OpenSTA
Power	OpenSTA

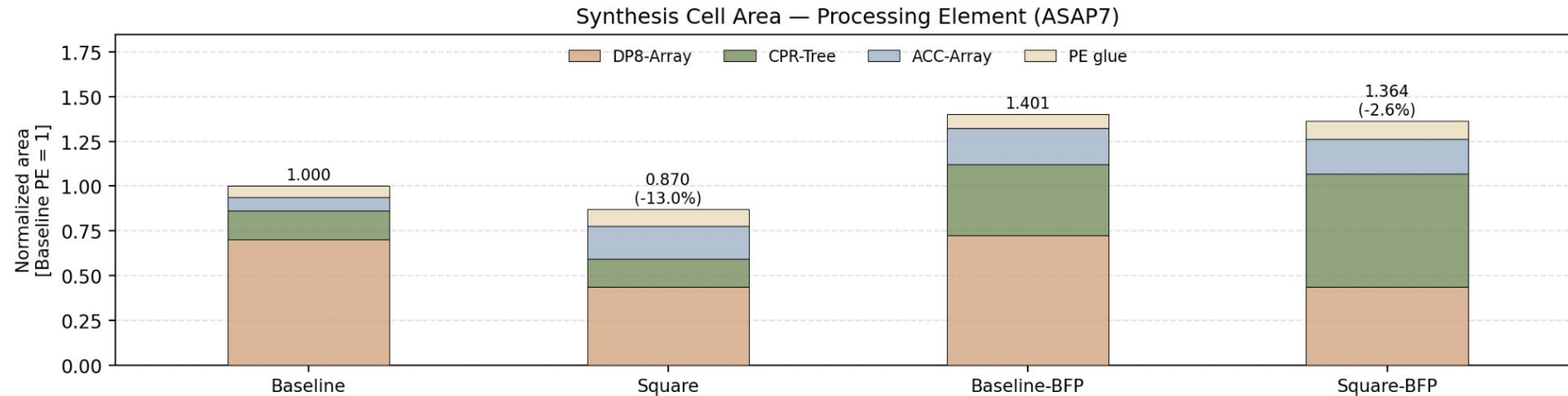
Area at the DP-Level

- **Scope:** the array of DP8 (8x4) inside one PE — the arithmetic units, nothing else
- The **squarer** is ~38 % smaller than the **Booth Radix-4 multiplier**
- **BFP costs nothing here** — the BFP variants reuse the integer units unchanged. The entire overhead is downstream, in the compression and accumulation stages



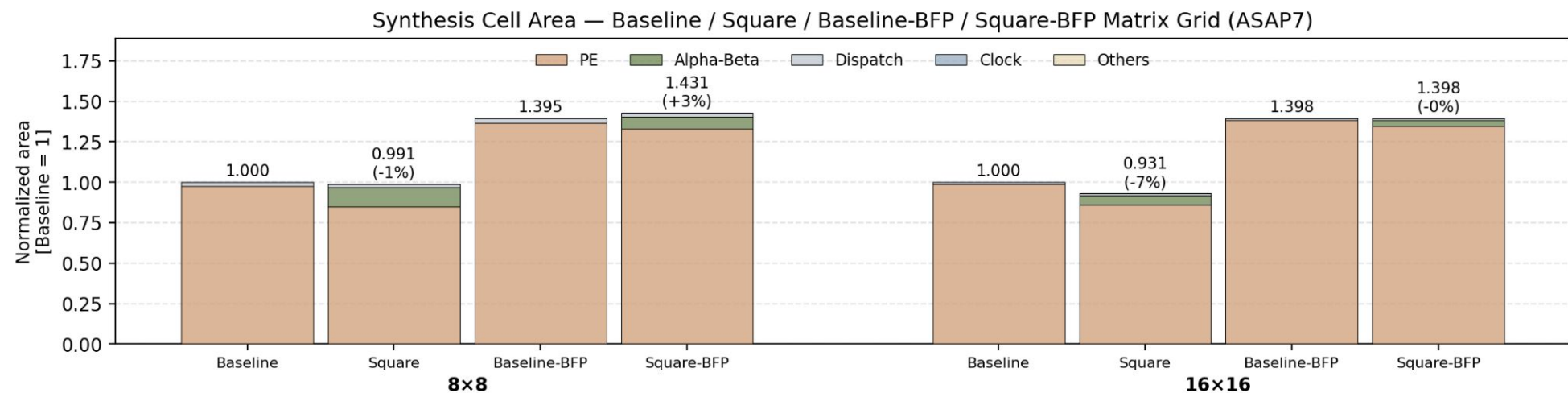
Area at the PE-Level – where the saving goes

- The **-38 %** core saving becomes **-13 %** at **PE-Level**: the squarer's **reconstruction** moves work into the **accumulator**, which grows **~2.4×**
- In **BFP** the dilution is stronger still — the **reconstruction** has to happen at a **common exponent scale**, which loads the reduction tree, leaving **-2.6 %**
- **BFP** inflates the **PE** by **~40 %**, almost entirely in the compression and accumulation stages



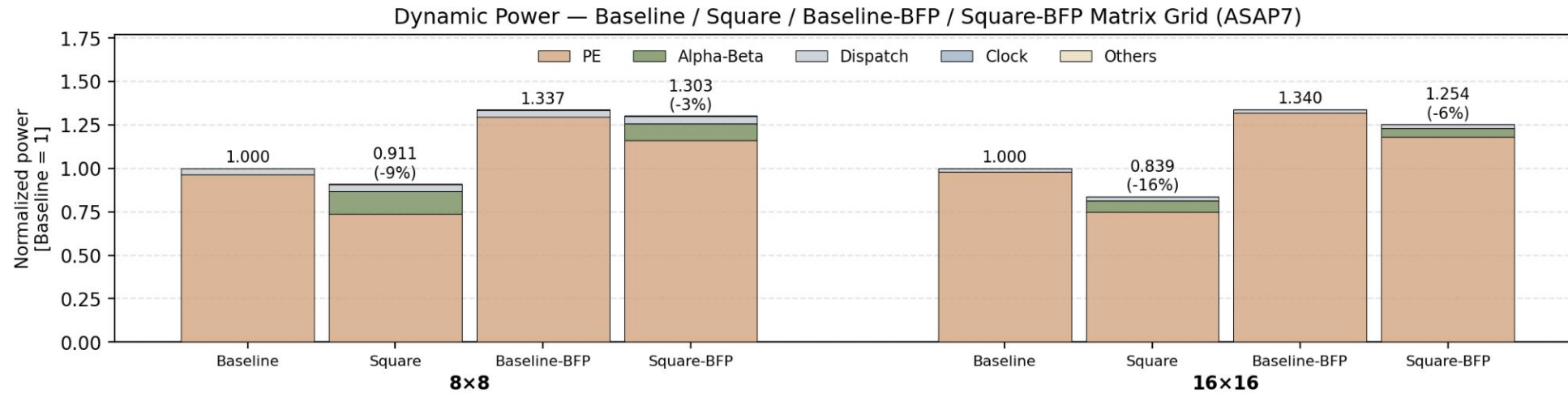
Area at the Grid-Level – the α/β tax appears

- **Scope:** the complete $N \times N$ grid, at 8×8 and 16×16 — the α/β generators appear as a new cost
- At 8×8 the two terms almost exactly cancel: -1 % (square), +3 % (square-BFP)
- At 16×16 the N^2 saving pulls ahead: -7 % (square), and square-BFP reaches parity
- Everything that is neither PE nor α/β — dispatch, clock, control — stays under 3 %



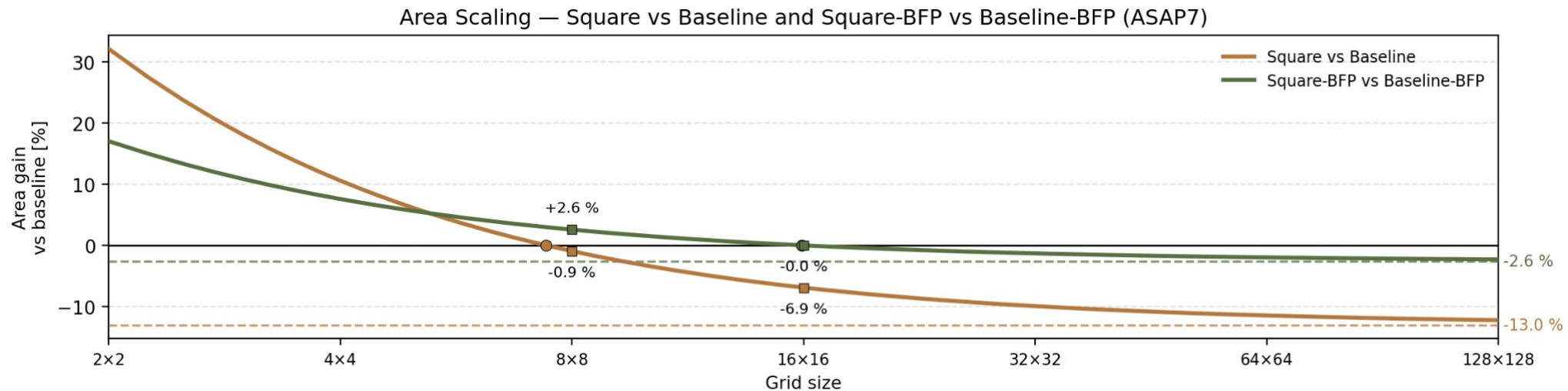
Power at the Grid-Level – a larger margin than area

- Scope: the complete N×N grid, at 8×8 and 16×16 — the α/β generators appear as a new cost
- 8×8: -9 % (square), -3 % (square-BFP) · 16×16: -16 % (square), -6 % (square-BFP)
- The margin is roughly double the area margin at every size
- Replacing multipliers with squarers removes more toggling than it removes gates



Area scaling – the $O(N^2)$ saving against the $O(N)$ tax

- % area gain against the matching baseline, as the grid grows
- Crossover: $N \approx 7.4$ (square) and $N \approx 15.9$ (square-BFP) — below it the shared hardware is not yet amortized
- Asymptotes: -13.0 % and -2.6 % — exactly the per-PE gains from the PE-level chart



Power scaling – earlier crossover, larger gain

- % power gain against the matching baseline, as the grid grows
- Crossover: $N \approx 4.9$ (square) and $N \approx 6$ (square-BFP) — both earlier than area
- Asymptotes: -23.7 % and -10.4 % — roughly double the area asymptotes
- Between the power and area crossover, the square buys energy at no area cost

